Implementing recursion: proposal

Status: accepted; the complete plan is the target, implemented in vertical slices. This is the design for Scale word recursion, book recursion, and F# package classification over IPC. The D194 book group and D189 callable interim: the complete runtime replaces the refusal with real recursive groups. The book branch and the word branch share one execution substrate; the callable/call contract is in `docs/callable-contract.typ`, the concrete substrate and versioning are in `docs/recursion-design.typ`, and the dependency graph is `docs/backlog-graph.typ`.

Objective

Implement Scale's recursion handling as five coordinated mechanisms, so that Scale words can express F# recursion semantics faithfully, with a two-tier classification path for F# packages reached over IPC. Preserve F#'s own diagnostics. Make every declaration auditable and checkable.

Scope

- Scale's own word definition semantics (`logic`, `cycle`, `and`).
- A classification record (abstraction) consumed by Scale's definition words, by books, and by the IPC bridge.
- A two-tier classification pipeline for F# packages: automatic (default) and explicit (opt-in), with the classifier as a build-time tool.
- Our own F# abstraction: a small, versioned representation of the F# subset used by the Prelude and by F# packages, lowered to the same execution substrate.
- Recursive binding groups in the package/image abstraction: units, arity, captures, initialization order, and a depth guard.
- Diagnostics surfaced through Scale's own diagnostic layer.

Separate layers, not deferred features: the IPC transport itself, F# package execution (the bridge carries references, not bodies), and quotation serialization. The complete classification and metadata path is in scope.

Five mechanisms

1. Separate word definition from value binding.

Keep `define` (word) and `bind` (variable) distinct. Recursion applies to word definitions. It does not apply to `bind`: that is a value binding and follows its own rules.

2. Explicit recursion at the definition site.

Every Scale word is defined by one of two plain words:

- `logic name { ... }` — non-recursive; the name is not in scope in the body.
- `cycle name { ... }` — recursive; the body calls the plain name.
- `cycle first { ... } and second { ... }` — mutually recursive; every group name is in scope in every body.

There is no `recurse` word. The plain name is the recursive reference. The group is the unit of recursion, and a `logic` self-call is a named refusal.

3. Initialization as a first-class phase.

Image construction proceeds in declared stages. A checker verifies, before execution begins, that all word definitions and referenced values are initialized in an order consistent with their declarations. This turns F#'s runtime lazy-init check into a construction-time diagnostic.

4. Scale-native diagnostics.

Scale emits its own diagnostics for recursion-related issues, with Scale source positions. F#'s diagnostics (FS0040, unused-`rec`, value restriction) are consumed and re-presented through the Scale diagnostic layer, not shown raw. Example message: "Word 'foo' is defined recursively but its body references uninitialized value 'bar'."

5. Two-tier classification for F# packages.

Tier A — automatic (default). F#'s front-end classification is read off directly. F# already assigns `let`, `let rec`, or `let rec ... and ...` to every binding; this classification **is** the package's semantics. No manifest required.

Tier B — explicit (opt-in, per package). A package manifest declares the recursion kind per binding, with an optional `reason` field. Used when initialization is cross-module or otherwise not derivable from F# read-off. The manifest is version-controlled with the package.

Hard rule for Tier B: it must still run F# read-off and compare. Explicit declarations override FCS; they do not replace it. Consequences:

- F#'s diagnostics remain available.
- Every explicit declaration becomes one of: **agrees with FCS** (silent), **disagrees with FCS** (diagnostic), **disagrees with declared reason** (accepted, tracked).
- Overriding disagreement without a `reason` is a build error.

The execution substrate

Every recursive mechanism lowers to one substrate. Recursion is a property of a **binding group**, not a special call opcode:

```
UnitGroup
    Name        : string
    Units       : Unit list
    InitOrder   : name list          // value definitions in order
    Classification : BindingClassification list

Unit
    Name        : string
    Arity       : int
    Captures    : slot list
    Instructions: Instruction list
```

- A call resolves a name in the current group/unit table; a self or mutual call is an ordinary call, not a special case.
- A unit call establishes the capture frame from the call arguments and the captured slots, then runs the unit's instructions.
- `MaxUnitDepth` bounds runaway recursion and reports a named failure.
- A group is one strongly connected component: a self-recursive function is a group of one; `let rec ... and ...` is a group of several; the initialization order is the group's `InitOrder`.
- A group is image/package data. The existing `LoweredProgram.Units`, `PackageUnit.Dependencies`, and `LoweredAbstraction.runProgram` are the starting point; the group record and the arity/capture call are the durable extension, not a parallel runtime.
- Books, `define` words, and the F# abstraction all lower to this substrate.

Our own F# abstraction

The Prelude is written in F#. Rather than mirror F#'s surface in Scale, define a small, versioned **F# abstraction** for the subset the Prelude and F# packages use:

- modules and bindings;
- `let`, `let rec`, and `let rec ... and ...` groups;
- functions with explicit arity and lexical parameters;
- calls, including higher-order calls;
- literals and pattern matching;
- source ranges and the classification record.

A build-time classifier reads F#'s own classification with FCS and emits the abstraction; a translator or a hand port lowers the same abstraction to `UnitGroups`. FCS is a **build-time** tool: the runtime and the AOT product consume the classification and never link FCS. That is the correct split, not a language limitation; the full F# read-off, diagnostics, and manifest are still implemented.

The abstraction is canonical and versioned. It is not a second runtime: it lowers to the same groups and units as book and `define` code.

Book recursion

A book is a module. By default all of its definitions form one recursive group. The loader computes the call graph and the group order; a function can call itself or any other definition.

The book surface uses `logic` for an ordinary definition and `cycle` for a recursive definition; `and` groups cycle definitions F#-style. A `logic` body must not call its own name. A `cycle` body may call any name in its group. An explicit marker must agree with the computed group; a disagreement is a diagnostic. D194 replaces the D188 refusal with real recursive groups; it is never a silent closure.

The quality bar

- **One substrate.** Books, `logic`/`cycle` words, the F# abstraction, and the Prelude port all use `UnitGroup`/`Unit`/the callable contract. No parallel recursion path.
- **The surface is complete.** `logic`, `cycle`, F#-style `and`, automatic book recursion plus optional markers, F# read-off, manifests, tiers, diagnostics, image stages, and IPC metadata all land.
- **No scope cut for code size.** The implementation is sliced for review, not reduced. FCS is build-time; the runtime is AOT and consumes metadata. That preserves the feature rather than removing it.
- **No silent success.** Every recursion and initialization failure is a named diagnostic with a source range; no closure-as-value fallback.
- **Every feature has a contract.** Each slice lands its documentation, tests, implementation, specification, and versioning together.

The abstraction contract

Both tiers converge on one record:

```
BindingClassification
    Name         : string
    Kind         : NonRecursiveFunction
                 | RecursiveFunction
                 | MutuallyRecursiveFunction of string list
                 | NonRecursiveValue
                 | RecursiveValue
    CallsInBody  : Set<string>     // diagnostics only
    SourceRange  : range
    DeclaredReason : string option // only in Tier B overrides
```

The record has no tier field. Tier is a property of the package, not the record.

Three package modes

- **Pure automatic** — no manifest; FCS read-off only. Default.
- **Mixed** — automatic plus per-binding overrides. Per-binding, not package-wide.
- **Strict explicit** — manifest required for all bindings; build fails on any binding without a declaration.

All three emit the same record and the same Scale word markers. Tier selection is a per-package annotation, not a language fork.

Deliverables

1. `logic` and `cycle` syntax and parser support, including `and` groups and plain-name recursion.
2. A build-time check: a `logic` body may not call its own name; a `cycle` group must call a group word.
3. `BindingClassification` type, versioned, with the FCS read-off adapter.
4. Manifest schema and parser (JSON or equivalent; specify the format in the design record).
5. Validation pass: explicit-vs-FCS comparison with the three outcomes above.
6. Scale diagnostic layer: consume F# diagnostics (FS0040, unused-`rec`, value restriction) with the F# source range mapped to the Scale source range.
7. Image construction phase with staged initialization and construction-time
8. checks.
9. IPC bridge: sends `(name, kind, version)`, never function bodies.
10. A recursive `UnitGroup`/`Unit` substrate with arity, captures, group
11. initialization order, and a depth guard.
12. Our own F# abstraction and a build-time classifier/translator for the Prelude
13. and F# packages.
14. Automatic book recursion plus the optional `logic`/`cycle` markers; the loader
15. computes the group and checks an explicit marker against it.
16. Versioning for the group, the classification record, the F# abstraction, and
17. the image/package schema.

Acceptance criteria

For each, provide a regression test that fails without the implementation:

- `let f x = f x` classified as `NonRecursiveFunction` (F# shadowing rule).
- `let rec f x = f x` classified as `RecursiveFunction`.
- `let rec f g = g f` classified as `NonRecursiveFunction` (non-recursive through the argument).
- `let rec f g = g f and h x = f h` classified as `MutuallyRecursiveFunction ["f"; "h"]`.
- `let rec x = 1 :: x` classified as `RecursiveValue`.
- A `logic` body that calls its own name is refused; a `cycle` body uses the plain name and runs.
- A Tier B manifest declaring `let-rec` for a binding FCS classifies as `let`, without `reason`, fails the build.
- A Tier B manifest with `reason` for the same case passes the build and produces the accepted-override diagnostic.
- F# FS0040 maps to a Scale diagnostic with the correct Scale source range.
- The IPC bridge serializes `(name, kind, version)` and refuses to serialize a body.
- A self-recursive unit executes through the group substrate and is bounded by the depth guard.
- A mutually recursive group executes; every unit sees the same group.
- A recursive group initializes value definitions in `InitOrder`; an out-of-order reference is a named diagnostic.

- A book with no marker is automatically recursive; a book with an explicit marker that disagrees with the computed group is a diagnostic.
- The F# abstraction round-trips and lowers to the same `UnitGroup`/`Unit` shape as a book or a `logic` word.
- The runtime assembly contains no FCS reference; the classifier is a build tool.
- A version skew between the group, classification, and F# abstraction is refused, never guessed.

Design records required before implementation

- **Manifest format and versioning** (schema, how overrides are keyed, how reasons are tracked).
- **Diagnostic mapping table** (every relevant F# warning/code to a Scale diagnostic, with the source-range translation rule).
- **Image construction stages** (what defines a stage, what the checker verifies, what a stage's failure produces).
- **Versioning of the record** (how the classification record evolves; how IPC consumers handle version skew).
- **UnitGroup/Unit schema** (the group record, arity/captures, the image and package encoding, the depth guard, and the schema version).
- **The F# abstraction subset** (the exact F# nodes the Prelude and packages use, the round-trip, and the lowering to groups/units).
- **The book surface** (`logic`/`cycle` groups, F#-style `and`, and how a marker is compared with the computed group).
- **The callable/group invocation** (the resolver, the arity/effect checks, and the relation to `docs/callable-contract.typ`).
- The IPC bridge does not execute F# code. It carries references.
- Scale does not reimplement F#'s compiler. It reads F#'s classification, or accepts an override with a reason.
- The value restriction is named in the record (`RecursiveValue`), but the semantics — reject, defer, or accept F#'s runtime check — is a Scale-side decision, not resolved by this proposal.
- Cross-package recursion is not globally visible to any single layer. Each package classifies itself. The proposal makes no claim about whole-program recursion soundness.

Resolved decisions

- The construction-time initialization checker lives in the **build-time image/F# classifier**, not in the runtime. The runtime consumes the checked group and never re-derives initialization.
- A `cycle ... and ...` group's ordering is part of `UnitGroup.InitOrder`; it does not require a second stage system beyond the group.
- `DeclaredReason` is an **enumerated** set with an `other` fallback that requires review. It stays auditable and extensible.
- FCS is a build-time dependency. The AOT runtime never links it; it consumes the versioned classification and group metadata. This is a separation of build and runtime, not a reduction of the feature.

The one-line summary

Read F#'s classification; let it be the default. Allow an explicit manifest to override it per package, but only with a declared reason that the build checks. Lower every classification

to a Scale `logic` or `cycle` definition. A `logic` self-call is refused; a `cycle` body uses the plain name. Surface F# diagnostics through Scale's own diagnostic layer. Never serialize function bodies across IPC — only `(name, kind, version)`. Books are recursive through `cycle`, with `and` grouping and the markers checked against the computed group. Every recursive form lowers to one `UnitGroup`/`Unit` substrate, and FCS remains a build-time tool.